Fr. C. Rodrigues Institute of Technology, Vashi

Department of Computer Engineering

CELBC710 - Computational Intelligence Laboratory | Semester VII | AY 2026-2027

Experiments 9 and 10

System Integration & Deployment, and Evaluation, Responsible AI & Documentation

Final Project Report: SymbioForge, an Intelligent Clinical Decision Support System for Cardiovascular Disease

Tools: Python, XGBoost, NumPy/PyTorch, Pydantic, Streamlit, pytest

1. Objective and Summary

To integrate the eight earlier experiments (knowledge representation, rule-based expert system, state-space search, data preparation, feature engineering, machine learning, deep learning and explainable AI) into one working prototype, and to evaluate it for performance, reliability, responsible use and deployment readiness.

SymbioForge is not a single classifier. It is a team of independent AI workflows under one orchestrator. A user describes their symptoms in a guided chat; each workflow examines the answers from its own angle (statistical model, deep network, expert rules, explanation, trust, what-if sensitivity, test planning); the orchestrator merges the evidence into a patient-level overview that explains what stands out, how sure the system is, and what to do next. A reviewer-facing view exposes the machinery behind it.

Decision support only, not a diagnosis. The prototype is trained on a public research dataset and is not clinically validated.

2. System Overview: Eight Workflows, One Orchestrator

Workflow	From	What it does	Where
Intake agent	Task 4 data preparation	Validates 13 inputs, flags missing optional tests and fills them with the training mode	cdss/schemas.py, features.py
Feature agent	Task 5 feature engineering	Builds the 12 model features in one fixed order from one formula module	cdss/features.py
Risk model	Task 6 machine learning	XGBoost probability of disease (primary verdict)	artifacts/xgb.json
Second-opinion model	Task 7 deep learning	Dense 64-32-16 DNN; flags disagreement with XGBoost	artifacts/dnn.json
Clinical rules agent	Tasks 1-2 knowledge + rules	Rules R1-R7, three-valued: fired / not fired / unknown	cdss/rules.py
Test planner	Task 3 state-space search	A* picks the cheapest tests that make the rules decisive; ML sensitivity ranks them	cdss/advisor.py
Explainer	Task 8 SHAP	Exact per-feature contributions, grouped into body-system domains	cdss/engine.py
Trust agent	Task 8 trust score	Confidence band from validation accuracy plus a 90% conformal set	cdss/engine.py
What-if agent	Task 8 counterfactuals	Model sensitivity over plausible changes; curves for key measurements	cdss/orchestrator.py
Synthesizer	Task 9 integration	Combines everything into the overview, doctor questions and report	cdss/orchestrator.py

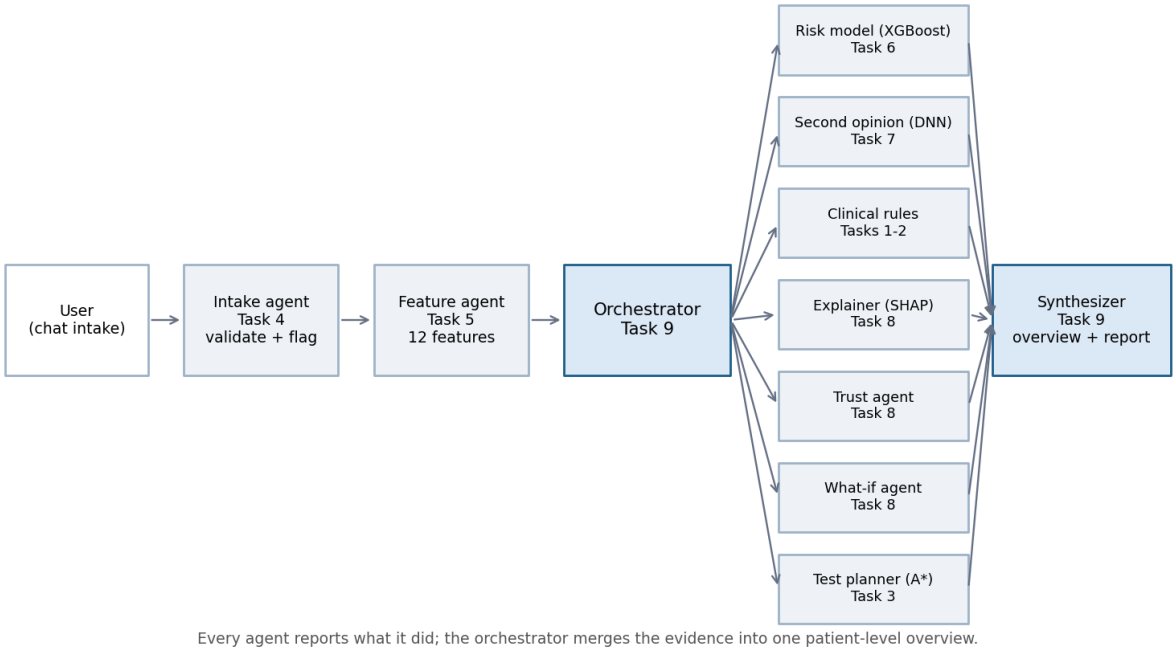


Figure 1. Orchestrated architecture. Each box is an independent workflow that reports what it did.

3. Implementation

3.1 Input validation and missing-data policy

- Pydantic schema with typed ranges (e.g. age 18-100, cholesterol 100-600 mg/dl). Bad input returns field-level errors; it never reaches a model.
- Zero cholesterol or blood pressure is rejected (disguised missing values in the training data).
- Optional tests (ST slope, vessel count, thallium result) may be missing. They are filled with the training mode, flagged, and the confidence band cannot be 'high' when two or more are missing. Rules are never evaluated on guessed values.

3.2 Feature engineering and models

The 12 features follow context.md exactly: cp_asymptomatic, exang, oldpeak_slope_risk, cardiac_risk_score, thalch, cp_atypical angina, oldpeak, chol, hr_reserve, rate_pressure_product, age, chol_age_ratio. Feature formulas live in one function that a parity test compares with the saved feature matrix (max difference under 1e-9), so training and serving cannot drift apart.

Data: 920 raw rows from four hospital sites, 918 after cleaning, prevalence 55.3%. Stratified split 70/15/15 (train 642, validation 138, test 138), seed 42. Imputer, scaler and models are fit on the training split only.

Model	5-fold CV AUC (train)	Test accuracy	Test F1	Test AUC
XGBoost(tuned)	0.876 +/- 0.031	0.819	0.837	0.896
LogisticRegression	0.874 +/- 0.033	0.833	0.852	0.904
SVM	0.859 +/- 0.027	0.819	0.834	0.879
RandomForest	0.853 +/- 0.034	0.841	0.853	0.870
GradientBoosting	0.853 +/- 0.031	0.833	0.854	0.893
XGBoost(context)	0.844 +/- 0.043	0.833	0.852	0.878
DecisionTree	0.811 +/- 0.025	0.819	0.847	0.855

XGBoost was selected by cross-validated AUC on the training split ({"colsample_bytree": 0.7, "eval_metric": "logloss", "learning_rate": 0.03, "max_depth": 2, "min_child_weight": 3, "n_estimators": 200, "random_state": 42, "reg_lambda": 2.0, "subsample": 0.8}). The DNN

(Dense 64-32-16, BatchNorm, Dropout, Adam 1e-3, early stopping) stopped at epoch 84. It is trained in PyTorch and served in plain NumPy from exported weights.

3.3 Explainability, trust and recommendations

- **Explanation:** exact TreeSHAP contributions per patient, summed into four domains (chest symptoms, stress test, cholesterol, overall profile) and shown as plain-language drivers.
- **Trust:** confidence band labelled from validation accuracy per confidence range, a 90% conformal prediction set, and disagreement flags (XGBoost vs DNN, ML vs rules).
- **What-if:** model sensitivity to plausible changes in cholesterol and blood pressure (capped at +/-30%), presented as model behaviour, not treatment advice.
- **Next-test advice:** A* over the missing tests (costs: slope 4, thallium 6, vessel count 8) plus a ranking by whether a result could flip the verdict.
- **Chat:** rule-based, no external AI service. It parses a free-text opening and answers follow-ups only from the computed analysis, so it cannot invent facts.

3.4 User interface

The user view is a guided chat: five short steps, plain-language questions, and a tooltip on every field explaining what it is and why it matters. Chest pain is classified from three yes/no questions rather than by asking for a medical term. For the demonstration the system is reached through a fictional clinic website (folder clinic/): its button redirects into the app, which links back to the clinic and its booking page; python run_demo.py starts both. The result page leads with a risk pattern, the estimated chance, confidence and information completeness; details sit in four tabs. A separate 'How it works' view serves reviewers.

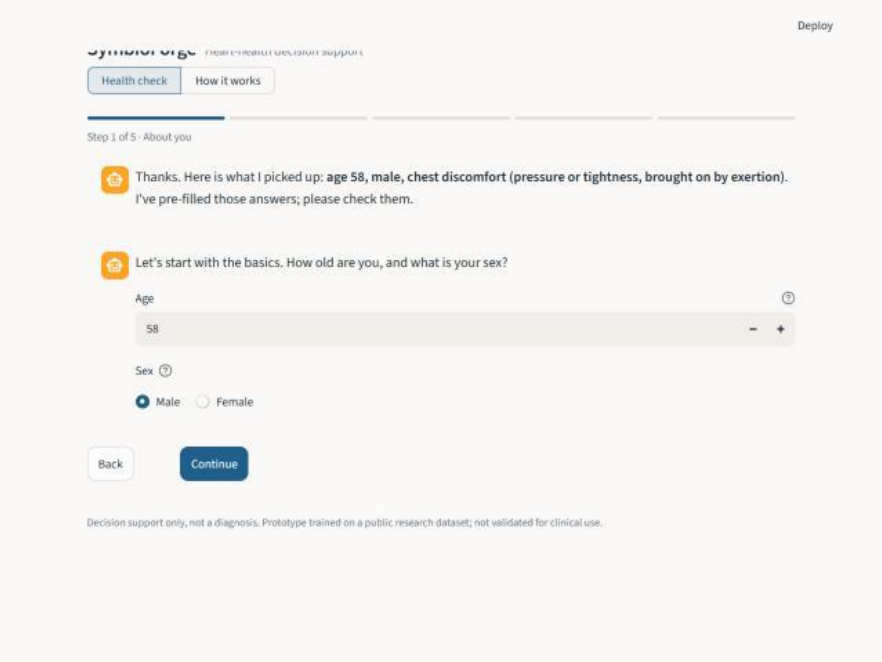


Figure 2. Guided chat intake with free-text pre-fill.

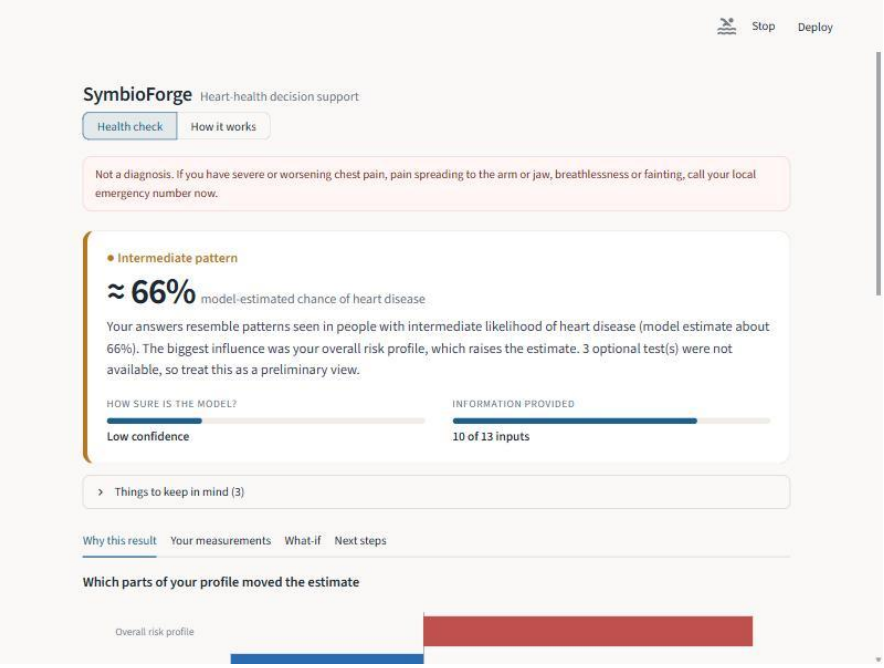


Figure 3. Overview: risk pattern, estimate, confidence and information completeness.

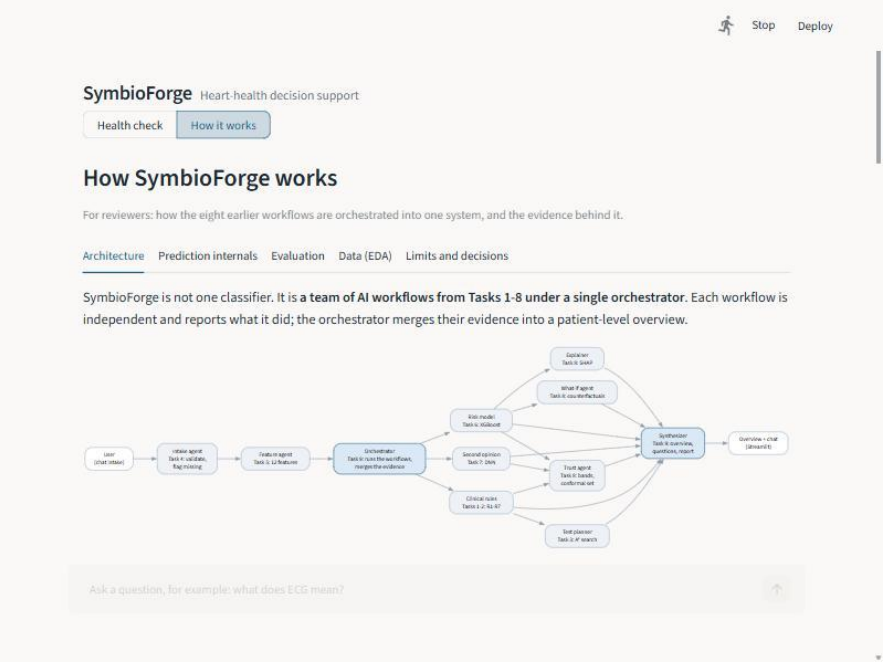


Figure 4. 'How it works' view: orchestration of the workflows for reviewers.

4. Evaluation Report

All figures are on the held-out test split (n = 138), untouched during training, tuning and calibration.

Model	Accuracy	Precision	Recall	ROC-AUC	Brier
xgb	0.819	0.831	0.842	0.896	0.124
dnn	0.819	0.849	0.816	0.897	0.125
mean_ensemble	0.826	0.842	0.842	0.897	0.123

AUC 95% bootstrap interval: XGBoost 0.84-0.95, DNN 0.84-0.95. The intervals overlap heavily, so the models cannot be separated statistically at this sample size. Averaging the two gave no meaningful gain (ensemble AUC 0.897), which is why XGBoost alone gives the verdict and the DNN acts as a second opinion. Always-predict-prevalence Brier baseline: 0.247.

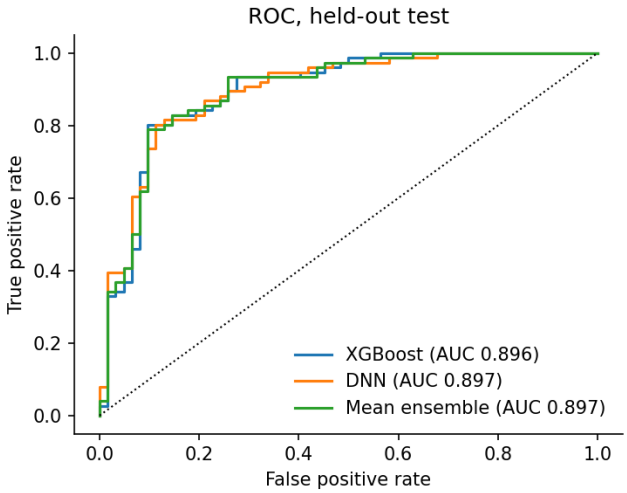


Figure 5. ROC curves, held-out test.

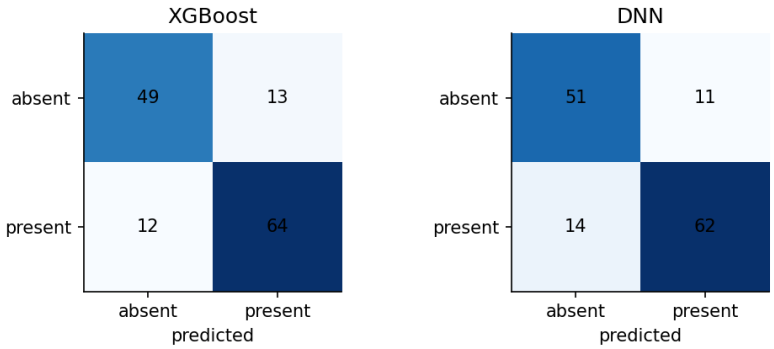


Figure 6. Confusion matrices at the 0.5 threshold.

4.1 Calibration, confidence and conformal prediction

Model confidence range	Test patients	Accuracy
0.50-0.65	23	0.391
0.65-0.75	20	0.950
0.75-0.85	32	0.844
0.85-0.95	50	0.920
0.95-1.00	13	0.923

Accuracy is clearly lowest in the least-confident band (0.391) and between 0.84 and 0.95 in the others. It is not strictly increasing, which is expected with such small bands, but it is why low-confidence results carry a 'defer to clinical judgement' warning. The 90% conformal set covered 92.0% of test patients (target 90%); 79% of patients received a single-label set and 21% the 'uncertain' two-label set. Bands contain only 13-50 patients each, so these figures are approximate.

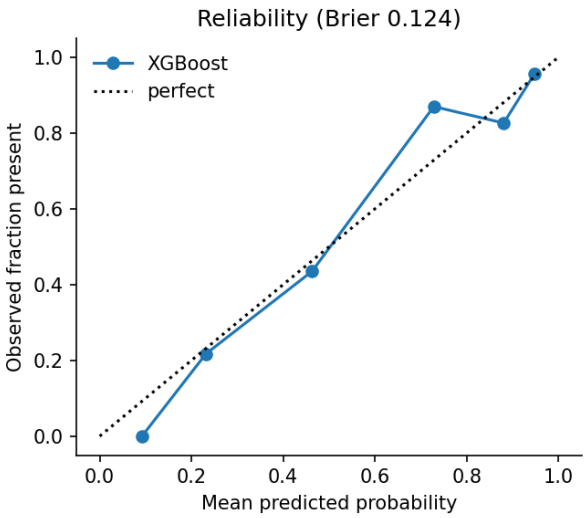


Figure 7. Reliability diagram for XGBoost.

4.2 Cross-checks: rule base and second model

The rule base reaches a decision for 59% of patients (the rest lack a needed test). Where it decides, ML accuracy is 0.921 when rules and ML agree and 0.526 when they disagree (n = 19). XGBoost and the DNN disagree on 6% of patients; XGBoost accuracy is 0.838 when they agree and 0.500 when they do not. Disagreement is therefore a useful warning, and the system lowers confidence when it occurs.

4.3 Effect of missing optional tests (ablation)

Subset	n	Accuracy	AUC
0 imputed	45	0.800	0.895
1 imputed	12	0.917	1.000
2+ imputed	81	0.815	0.884
all optional fields neutralised	138	0.819	0.895

Hiding the three optional tests from every test patient changes AUC only slightly, so the model leans mainly on symptoms and risk factors. The reliability warning is still kept, because the filled-in value can be wrong for an individual. Small subsets, so treat as indicative.

4.4 Explainability validation

Global SHAP ranks: cp_asymptomatic (0.61), cardiac_risk_score (0.51), oldpeak_slope_risk (0.22), oldpeak (0.19), exang (0.19). The dominant role of asymptomatic chest pain agrees with Tasks 1, 2, 5 and 8 (four independent methods). In the training data a 'silent' presentation was more often found to have disease, so the interface warns that no chest pain is not reassuring.

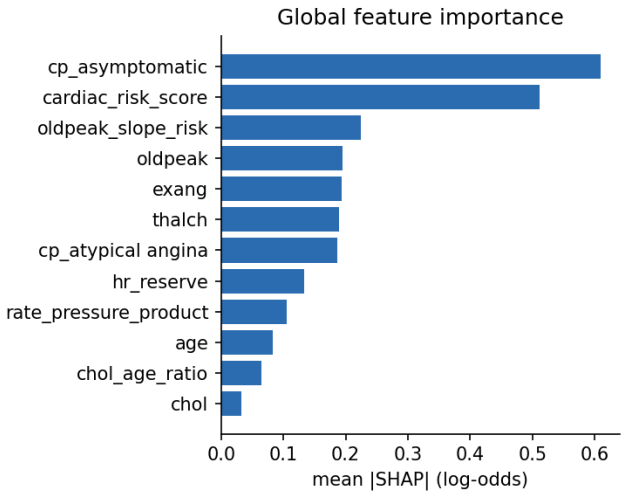


Figure 8. Global feature importance (mean |SHAP|, test set).

5. Testing Report

Area	What is checked	Result
Feature parity	Pipeline output equals the saved feature matrix (max diff < 1e-9); formulas hand-checked	Pass
Model parity	NumPy DNN reproduces the trained DNN's test AUC; XGBoost reproduces its reported AUC (+/- 0.005)	Pass
Rule engine	Three-valued logic, strict/inclusive boundaries (e.g. oldpeak > 2.8, age in [56,58]), R7 default	Pass
Validation	Out-of-range, wrong-type and zero-value inputs return field-level errors	Pass
End-to-end	All four scripted patients (higher risk, lower risk, borderline, rule-vs-ML disagreement) run through the full pipeline	Pass
Determinism	Same input gives identical output, including the counterfactual	Pass
A* planner	Returns the cheapest decisive test set (slope + vessel count, cost 12) for the borderline case	Pass
Cleaning	Zero-to-missing, duplicate removal, target binarisation, no constant filler for imputed cholesterol	Pass
Chat and glossary	Intro parsing, status cut-offs, every suggested question answered, emergency wording	Pass
Dashboard	Streamlit test runs the guided flow and every example through to the result page, and the How-it-works view	Pass

25 automated tests pass. Robustness: invalid inputs rejected 8/8; 300 random valid patients processed without error (300/300). Latency of the full pipeline including SHAP, counterfactual and planner: median 56 ms, 95th percentile 95 ms (target under 500 ms). An independent code review found six defects (a constant imputation value that acted as a hidden disease marker, miscalibrated confidence labels, an ambiguous headline probability, a spec mismatch, misleading 'actionable' counterfactuals and rule statistics that used imputed values); five were fixed and one was recorded as a deliberate design change.

Domain and user validation: scripted cases and automated checks only. No clinician or end-user study has been carried out; this is listed as future work.

6. Responsible AI Assessment

Area	Finding and mitigation
Intended use	Educational decision support for one patient at a time. Emergency and not-a-diagnosis notices appear on every screen and in the report.
Data	Public research data from four hospitals, collected decades ago. Not representative of any current population.
Missing data	Flagged, never silent. Two or more filled fields block a 'high' confidence band. Rules never run on guessed values.
Out-of-distribution	Age outside the 28-77 training range triggers a warning.

Transparency	SHAP per patient, plain-language reasons, rule trace, agreement chips, a visible trace of every workflow. The DNN has no per-patient explanation, and the UI says so.
Uncertainty	Validation-based confidence bands, conformal set, and disagreement flags. 'Uncertain' is an allowed output.
Counterfactuals	Labelled model sensitivity, restricted to cholesterol and blood pressure, capped at +/-30%; never presented as treatment effect.
Chat	Rule-based and offline; answers are assembled from computed evidence and cannot hallucinate. Emergency wording is always available.
Privacy	Stateless: no patient data stored or logged; no data leaves the machine; no external AI service.
Reproducibility	Fixed seeds, pinned split, saved artifacts, determinism test, one-command rebuild.

Fairness check (test set). Groups are small, so differences are weak evidence.

Group	n	AUC	Recall	Precision	Prevalence
sex: male	113	0.881	0.843	0.843	0.62
sex: female	25	0.895	0.833	0.714	0.24
age_band: >65	13	0.955	0.909	0.909	0.85
age_band: 56-65	47	0.802	0.871	0.794	0.66
age_band: 46-55	51	0.911	0.800	0.833	0.49
age_band: <=45	27	0.981	0.778	0.875	0.33
site: Cleveland	46	0.898	0.783	0.818	0.50
site: Hungary	53	0.897	0.800	0.762	0.38
site: Switzerland	18	0.941	0.882	1.000	0.94
site: VA Long Beach	21	0.825	0.938	0.789	0.76

Limitations: women are a small share of the data; disease prevalence varies strongly by hospital (for example 94% in one site against 36% in another), and hospital also drives which values are missing, so it acts as a hidden confounder; the test set is small; there is no external or temporal validation.

7. Requirements and Design Decisions

ID	Requirement
FR1-FR2	Validate raw inputs; handle optional/missing inputs explicitly
FR3-FR5	Build the 12-feature vector exactly; run XGBoost and DNN; verdict from XGBoost with disagreement flag
FR6	Evaluate rule base R1-R7, report fired/unknown rules and ML agreement
FR7-FR9	SHAP explanation; confidence band with reliability warnings; counterfactual sensitivity
FR10	Next-test recommendation by A* when key tests are missing
FR11-FR12	Readable summary and downloadable report; guided UI from inputs to overview
NFR	Latency under 500 ms (met), deterministic, stateless, one-command run, zero cost, every feature traceable to its formula

Key design decisions (full log in ARCHITECTURE.md):

Decision	Why
XGBoost gives the verdict; the DNN is a second opinion	The mean ensemble was no better, and SHAP explains XGBoost, keeping verdict and explanation consistent
Three-valued rules	A rule must not fire on a guessed value; the unknowns drive the test planner
Random draw instead of a constant for unobservable cholesterol	A constant filler became a hidden site marker the model learned (found in review)
DNN trained in PyTorch, served in NumPy	TensorFlow would not load on the development machine; no heavy runtime dependency
Confidence labels from validation accuracy	Fixed probability cut-offs ignored calibration
Rule-based chat, no external AI service	Zero cost, offline, cannot invent facts; trade-off is limited open-ended understanding

8. User Manual and Deployment Guide

8.1 Using the system

- Open the app. 'Health check' is for users; 'How it works' is for reviewers.
- Describe symptoms in your own words (optional) or press Start. Answer the five short steps; hover the '?' next to any field for a plain explanation.
- Read the overview: risk pattern, estimated chance, confidence and how much information was provided.
- Use the tabs: Why this result, Your measurements, What-if, Next steps (including questions for a doctor and a downloadable report).
- Ask follow-up questions in the chat, for example 'What does thallium mean?'. For a quick demo use the four ready-made examples on the first screen.

8.2 Running and deploying

Local run (PowerShell):

```
cd C:\Users\aarya\OneDrive\Desktop\CODING\SymbioForge
pip install -r requirements.txt
streamlit run app.py
```

Rebuild everything: pip install -r requirements-train.txt, then python train.py, python eda.py, python evaluate.py, python build_docs.py, python -m pytest. Cloud deployment: push the repository to GitHub and create a Streamlit Community Cloud app with main file app.py (Python 3.11+, requirements.txt only; no secrets and no database). Artifacts are under 300 KB and committed, and XGBoost is stored as portable JSON so it does not depend on the training machine.

Deployment readiness: all dependencies are free and open source; the app starts cold in about 15 seconds on a slow disk; no external services are required.

9. Deliverables Checklist

Task	Deliverable	Where
9	Application (running instance)	streamlit run app.py
9	Integrated source code	cdss/, ui/, app.py, train.py, evaluate.py, eda.py
9	Dashboard	Health check view and How it works view
9	Clinic front-door demo	clinic/ (static demo site that redirects into the app); run_demo.py
9	Report generator	cdss/report.py; download button; reports/sample_report.html
9	Deployment documentation	README.md and Section 8 of this report
10	Evaluation report	Section 4; reports/evaluation.json
10	Responsible AI report	Section 6
10	User manual	Section 8.1
10	SRS / SDD	Section 7, SPEC.md, ARCHITECTURE.md
10	Testing report	Section 5; tests/test_core.py
10	Final project report	This document
10	Deployment guide	Section 8.2

10. Conclusion and Future Enhancements

The experiments are integrated into one system whose value lies in the combination: eight workflows produce independent evidence, an orchestrator reconciles them, and the patient receives an explained, uncertainty-aware overview rather than a bare label. On the held-out test split XGBoost reaches 81.9% accuracy and 0.896 AUC, the system answers in about 56 ms, and all 25 automated tests pass. The prototype is honest about its limits: small test set, historical single-source data, no clinical validation.

Future work: repeated cross-validation and site-held-out validation; calibrated probabilities (isotonic); a clinician and user study; an optional language-model layer for the chat, restricted to the computed analysis; and multilingual support.